

Task 2 (Intermediate Level)

Web Scraper & Hangman — Python Solutions

1. Web Scraper

Extracts quotes, authors and tags from every page of **quotes.toscrape.com** (a public sandbox site built for scraping practice) using **requests** and **BeautifulSoup**, then saves the results to a CSV file.

Setup: pip install requests beautifulsoup4

```
"""
Task 2.1 - Web Scraper
Extracts data (titles and quotes) from a website using BeautifulSoup.
Target site: https://quotes.toscrape.com (a site built for scraping practice)
"""

import requests
from bs4 import BeautifulSoup
import csv

def scrape_quotes(base_url="https://quotes.toscrape.com"):
    """Scrape all quotes, authors and tags across every page of the site."""
    all_data = []
    url = base_url
    page = 1

    while url:
        print(f"Scraping page {page}: {url}")
        response = requests.get(url, timeout=10)
        response.raise_for_status()

        soup = BeautifulSoup(response.text, "html.parser")
        quote_blocks = soup.find_all("div", class_="quote")

        for block in quote_blocks:
            text = block.find("span", class_="text").get_text(strip=True)
            author = block.find("small", class_="author").get_text(strip=True)
            tags = [tag.get_text(strip=True) for tag in block.find_all("a", class_="tag")]

            all_data.append({
                "quote": text,
                "author": author,
                "tags": ", ".join(tags)
            })

        # Find the "Next" button to move to the next page
        next_btn = soup.find("li", class_="next")
        if next_btn:
            next_href = next_btn.find("a")["href"]
            url = base_url.rstrip("/") + "/" + next_href.lstrip("/")
            page += 1
        else:
            url = None

    return all_data

def save_to_csv(data, filename="quotes.csv"):
    with open(filename, "w", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["quote", "author", "tags"])
        writer.writeheader()
        for item in data:
            writer.writerow(item)
```

```

"""Write the scraped data to a CSV file."""
if not data:
    print("No data to save.")
    return

with open(filename, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["quote", "author", "tags"])
    writer.writeheader()
    writer.writerows(data)

print(f"Saved {len(data)} quotes to {filename}")

if __name__ == "__main__":
    scraped_data = scrape_quotes()
    save_to_csv(scraped_data)

# Print a small preview
for item in scraped_data[:5]:
    print(f"\n\"{item['quote']}\\" - {item['author']} [{item['tags']}]")

```

2. Hangman

A word-guessing game that runs in the terminal. It shows the player's progress as blanked-out letters, draws an ASCII-art gallows that grows with each wrong guess (visual progress), and offers a one-time hint at the cost of a guess.

```
"""
Task 2.2 - Hangman
A word-guessing game with ASCII-art visual progress and a hint system.
"""

import random

WORDS = {
    "python": "A popular programming language named after a comedy group.",
    "hangman": "The name of the game you are currently playing.",
    "developer": "Someone who writes software.",
    "keyboard": "You type on this.",
    "function": "A reusable block of code.",
}

HANGMAN_PICS = [
    """
    -----
    |       |
    |       |
    |       |
    |       |
    -----
    """,
    """
    -----
    |       |
    |       |
    |       |
    |      0
    -----
    """,
    """
    -----
    |       |
    |       |
    |      0
    |      |
    -----
    """,
    """
    -----
    |       |
    |      0
    |      |
    |     /|
    -----
    """,
    """
    -----
    |       |
    |      0
    |     /|\
    -----
    """
]
```

```

-----
"""
"""

    |
    | 0
    | /|\
    | /
    |
-----
"""
"""

    |
    | 0
    | /|\
    | / \
    |
-----
"""
,
]

MAX_ATTEMPTS = len(HANGMAN_PICS) - 1

def choose_word():
    return random.choice(list(WORDS.items()))

def display_progress(word, guessed_letters):
    return " ".join(letter if letter in guessed_letters else "_" for letter in word)

def play_hangman():
    word, hint = choose_word()
    guessed_letters = set()
    wrong_guesses = 0
    hint_used = False

    print("Welcome to Hangman!")
    print(f"The word has {len(word)} letters. Type 'hint' for a clue.\n")

    while wrong_guesses < MAX_ATTEMPTS:
        print(HANGMAN_PICS[wrong_guesses])
        print("Word: " + display_progress(word, guessed_letters))
        print(f"Wrong guesses left: {MAX_ATTEMPTS - wrong_guesses}")
        print(f"Guessed so far: {'', '.join(sorted(guessed_letters)) or 'none'}")

        guess = input("\nGuess a letter (or 'hint'): ").strip().lower()

        if guess == "hint":
            if not hint_used:
                print(f"Hint: {hint}")
                hint_used = True
                wrong_guesses += 1 # using a hint costs one wrong guess
            else:
                print("You've already used your hint!")
                continue

        if len(guess) != 1 or not guess.isalpha():
            print("Please enter a single letter.")
            continue

        if guess in guessed_letters:
            print("You've already guessed that letter.")
            continue
        guessed_letters.add(guess)

```

```
    if guess in word:
        print("Correct!")
        if all(letter in guessed_letters for letter in word):
            print(HANGMAN_PICS[wrong_guesses])
            print(f"\nYou won! The word was '{word}'.")
            return
    else:
        wrong_guesses += 1
        print("Wrong guess!")

    print(HANGMAN_PICS[wrong_guesses])
    print(f"\nGame over! The word was '{word}'.")

if __name__ == "__main__":
    play_again = "y"
    while play_again == "y":
        play_hangman()
        play_again = input("\nPlay again? (y/n): ").strip().lower()
    print("Thanks for playing!")
```